Softwareentwicklung mit Java 2



3. Interfaces

Vorschau: Generics

= Variable mit Typ als Wert

Näheres zu Generische Typen mit Typparameter in Kapitel 5. Generics

Generische Typen verwenden Typparameter als Platzhalter für Typen

Beispiel: generische Liste mit Typparameter **E** für den Typ der Elemente

- **E** steht dabei für einen beliebigen Typ
- und wird als **Typ für Parameter** beim Konstruktor und bei **add**
- und als **Typ für Rückgabewert** von **get** verwendet

```
public class List<E> {
  public List(E...initial) {...}
  public E get(int idx) {...}
  public boolean add(E elem) {...}
  ...
}
```

- ☐ Bei konkreten Objekten wird der **Typparameter** auf einen **konkreten Typ gesetzt**

```
List<Student> students = new List<Student>();
```

E wird auf **Student** gesetzt

- ☐ Damit sind **Zugriffe typsicherer**

```
students.add(new Student("Hans"));
Student hans = students.get(0);
```

nur **Student** anfügen erlaubt

garantiert **Student** retour

```
students.add(new Professor("Wirth"));
```

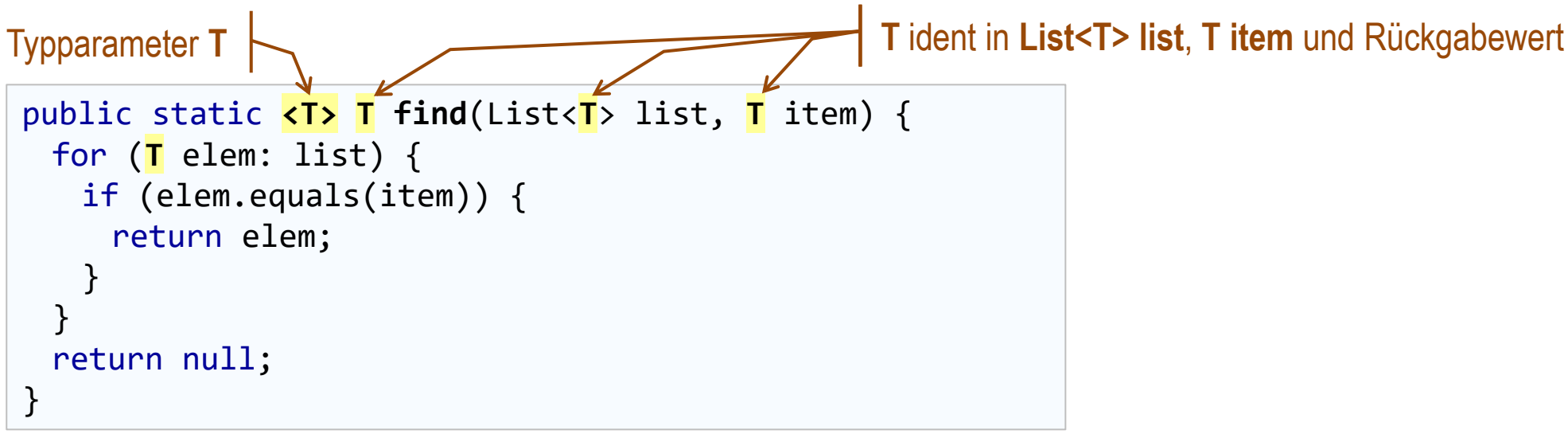
✗ **Compilerfehler**

© H. Prähofer, 2026

Vorschau: Generics

Methodendeklaration mit Typparameter

- **Typparameter** zum Bestimmen der Typen der **Parameter**, **Variablen** und **Rückgabewerten**




```
public static <T> T find(List<T> list, T item) {
  for (T elem: list) {
    if (elem.equals(item)) {
      return elem;
    }
  }
  return null;
}
```

- Ableitung des **konkreten Typs** aus den **aktuellen Methodenparametern**

```
List<String> keywords = …;
String found = find(keywords, "lambda");
```

Typ **T = String**

```
List<Person> persons = …;
Person found = find(persons, new Person(…));
```

Typ **T = Person**

© H. Prähofer, 2026

3. Interfaces

3.1 Motivation

3.2 Interfaces in Java

3.3 Default-Methoden

3.4 Statische und private Members

3.5 Beispiele aus der Bibliothek

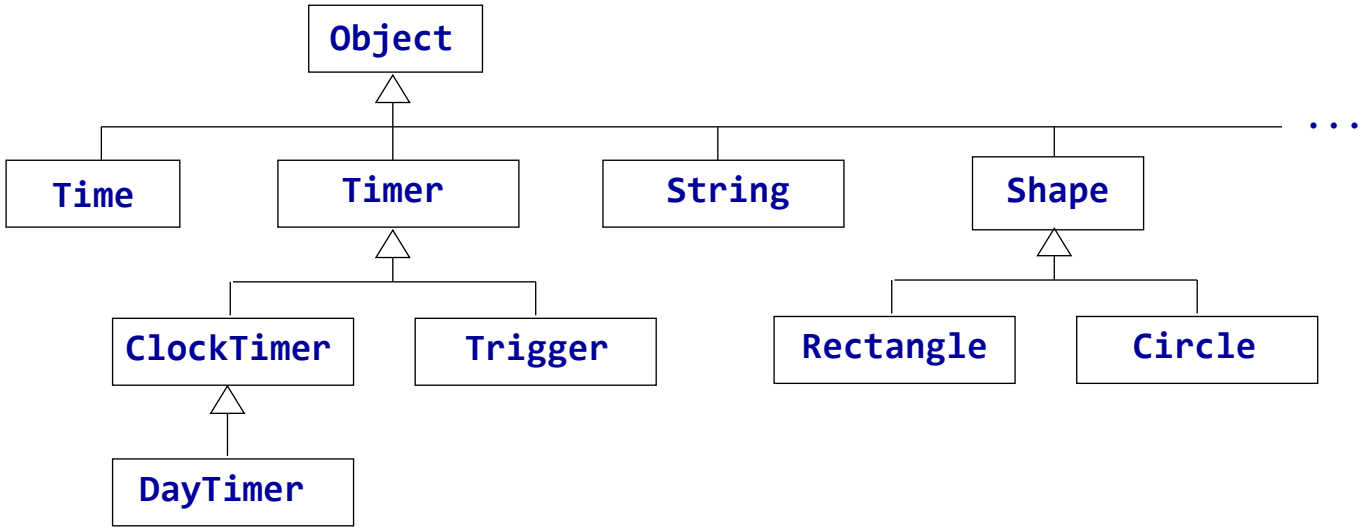
3.6 Fallstudie List

© H. Prähofer, 2026

Einfachvererbung bei Klassen (Single Inheritance)

- Java-Klassen unterstützen nur Einfachvererbung bei Klassen
  ➔ jede Klasse hat **genau eine Superklasse**

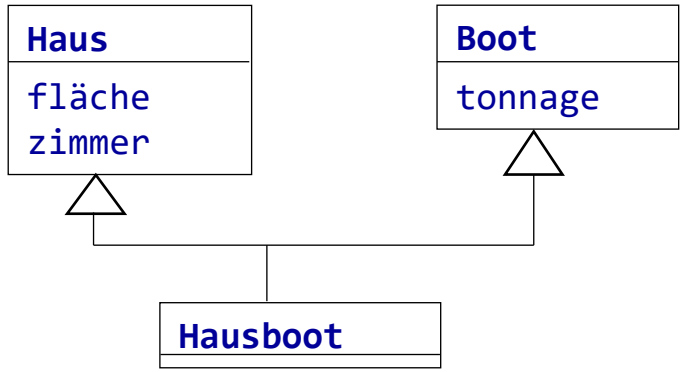
ausgenommen **Object**, die keine Superklasse hat



© H. Prähofer, 2026

Mehrfachvererbung bei Klassen (multiple inheritance)

Mehrfachvererbung heißt Ableiten einer Klasse aus mehreren Oberklassen

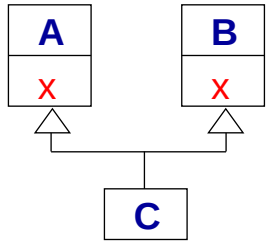


- *Hausboot* erbt alle Felder und Methoden von *Haus* und *Boot*
  ➔ ***Vererbung***

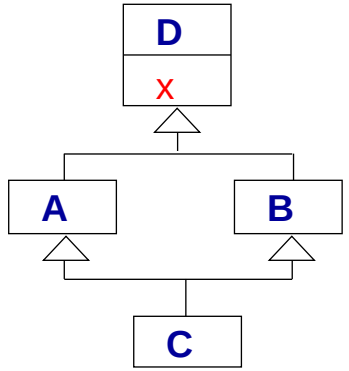
- *Hausboot* ist sowohl mit *Haus* als auch mit *Boot* kompatibel
  ➔ ***Typkompatibilität***

Probleme mit Mehrfachvererbung

- Namenskonflikte bei Feldern



2 Felder mit gleichem Namen in **C**



"Rautenstruktur"
Wird **x** einmal oder zweimal an **C** vererbt?

➔ **Mehrfache Vererbung bei Klassen wird von Java nicht unterstützt!**

© H. Prähofer, 2026

3. Interfaces

3.1 Motivation

3.2 Interfaces in Java

3.3 Default-Methoden

3.4 Statische und private Members

3.5 Beispiele aus der Bibliothek

3.6 Fallstudie List

© H. Prähofer, 2026

Interfaces in Java

Interfaces erlauben Mehrfachvererbung

- Klassen können mehrere Interfaces implementieren

Interfaces können haben

- abstrakte Methoden (**public abstract**)
- konkrete Default-Methoden (**public default**)
- private Methoden (**private**)

- statische Konstanten (**public static final**)
- statische Methoden (**public static**)
- private statischen Methoden (**private static**)

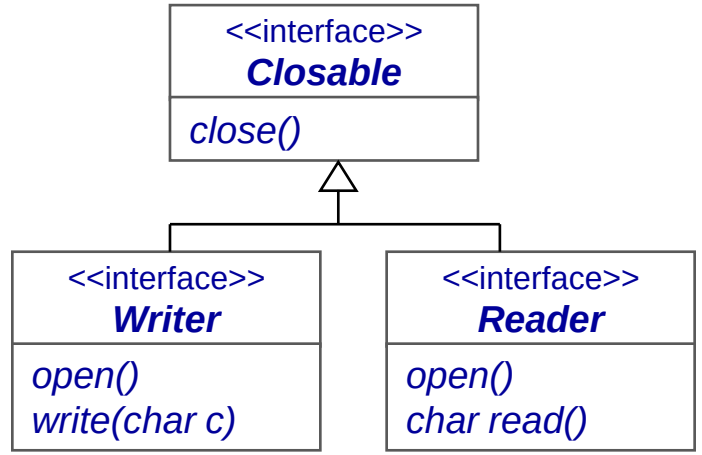
Unterschied zu (abstrakten) Klassen

- **keine** Objektfelder
- **keine** Konstruktoren
- **keine** normalen nicht-abstrakten Methoden

nur Default-Methoden

Interfaces sind Typen

Interfaces in Java



- Schlüsselwort **interface**
- abstrakte Methoden
  - implizit **public abstract**

```
public interface Closable {
  void close();
}
```

```
public interface Reader extends Closable {
  char read();
  void open();
  void close();
}
```

automatisch
public abstract

```
public interface Writer extends Closable {
  void write(char c);
  void open();
  void close();
}
```

(Default-, private und static Methoden später)

© H. Prähofer, 2026

Mehrfachvererbung bei Interfaces in Java

- Klassen können Interfaces implementieren
  - Schlüsselwort **implements**
  - eine Klasse kann **mehrere Interfaces implementieren** (aber nur eine Klasse erweitern)

Beispiel: Klasse **File** erweitert **OSObject** und implementiert Interfaces **Reader** und **Writer**



© H. Prähofer, 2026

Unified Modeling Language (UML)

Interface in UML

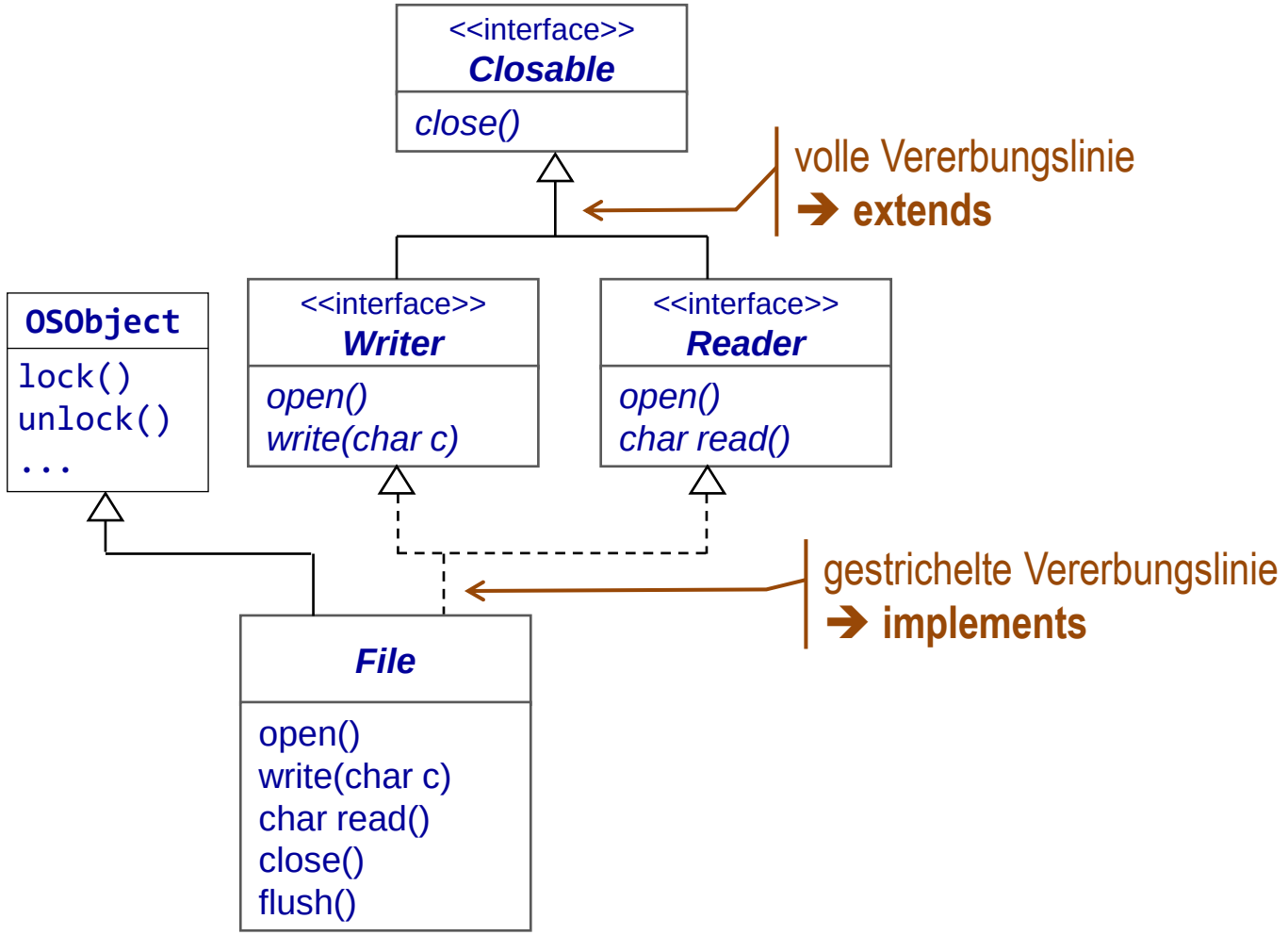
- Name kursiv
- abstrakte Methoden kursiv
- optional mit Annotation **<<interface>>**
- keine Felder, nur Methoden

Erweiterung von Interfaces

- wie bei Klassenvererbung

Implementierung von Interfaces durch Klassen

- mit gestrichelten Linie

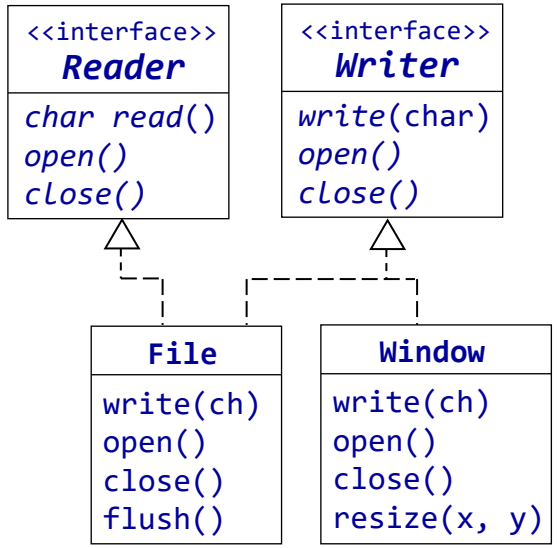


© H. Prähofer, 2026

Interfaces als Typen

Interfaces sind Typen

```
Writer writer;
```



- Polymorphe Zuweisung und dynamische Bindung

```
writer = new File();
writer.open();
writer.write(c);
writer.flush();
```

```
writer = new Window();
writer.open();
writer.write(c);
writer.resize(200, 200);
```

- Unterschiedliche Verwendung eines Objekts

```
File file = new File();
```

```
Reader reader = file;
reader.open();
char c = reader.read();
reader.close();
```

file als Reader

```
Writer writer = file;
writer.open();
writer.write(c);
writer.close();
```

file als Writer

© H. Prähofer, 2026

3. Interfaces

3.1 Motivation

3.2 Interfaces in Java

3.3 Default-Methoden

3.4 Statische und private Members

3.5 Beispiele aus der Bibliothek

3.6 Fallstudie List

© H. Prähofer, 2026

Default-Methoden

Default-Methoden sind Methoden mit konkreten Implementierungen

- werden mit **default** gekennzeichnet
- **verwenden** oftmals **abstrakte Methoden**

```
public interface Writer {
  public abstract void write(char c);
  public abstract void open();
  public abstract void close();

  default void writeLn() {
    write('\n');
  }

  default void write(String s) {
    for (char ch : s.toCharArray()) write(ch);
  }

  default void writeLn(String s) {
    write(s);
    writeLn();
  }
}
```

abstrakte Methoden

konkrete Methode **writeLn** verwendet abstrakte Methode **write**

konkrete Default-Methoden implementiert unter **Verwendung der abstrakten Methoden**

© H. Prähofer, 2026

Erben von Default-Methoden

Klassen müssen nur abstrakte Methoden implementieren

```
public class File implements Writer {
  public void write (char ch) {...}
  public void open () {...}
  public void close () {...}
  ...
}
```

Die **abstrakten Methoden** von **Writer müssen implementiert** werden!

und erben Default-Methoden

```
Writer writer = new File();
writer.writeLn("Hello World");
```

die Default-Methode **writeLn(String)** wird geerbt

© H. Prähofer, 2026

Anwendung Default-Methoden: "Rich Interfaces"

Schmale abstrakte Schnittstelle aber breite konkrete Implementierung

Beispiel: **Ordered<T>**

```
public interface Comparable<T> {
  public int compareTo(T other);
}
```

< 0 : this < other
> 0 : this > other
== 0 : this.equals(other)

```
public interface Ordered<T> extends Comparable<T> {
  // inherits int compareTo(T other);

  default boolean less(T other) {
    return this.compareTo(other) < 0;
  }
  default boolean lessEqual(T other) {
    return this.compareTo(other) <= 0;
  }
  default boolean greater(T other) {
    return this.compareTo(other) > 0;
  }
  default boolean greaterEqual(T other) {
    return this.compareTo(other) >= 0;
  }
}
```

eine abstrakt (geerbt)

4 Default

verwenden abstraktes **compareTo**

```
public class Fraction
    implements Ordered<Fraction> {
  private int num, denom;
  …
  public int compareTo(Fraction that) {
    return this.denom * that.num -
           that.denom * this.num;
  }
}
```

1 Methode implementiert

```
Fraction f1 = new Fraction(3, 4);
Fraction f2 = new Fraction(2, 3);

if (f1.less(f2)) {...}
else if (f1.greater(f2)) {...}
else if (f1.lessEqual(f2)) {...}
else if (f1.greaterEqual(f2)) {...}
```

4 Default-Methode geerbt

© H. Prähofer, 2026

Anwendung Default-Methoden: Schnittstellenerweiterungen

Default-Methoden zur Erweiterung bestehender Interfaces (ohne die Klassen ändern zu müssen)

Beispiel: Interface **Collection**

- **Collection** ist Basisklasse für alle Collection-Klassen in Java-Bibliothek
- Erweiterung von Collection um Default-Methoden zur Unterstützung der funktionalen Programmierung (siehe Kapitel Streams)

```
public interface Collection<E> extends Iterable<E> {
  int size();
  boolean isEmpty();
  boolean contains(Object o);
  boolean add(E e);
  boolean remove(Object o);
  ...
  default <T> T[] toArray(IntFunction<T[]> generator) {...}
  default boolean removeIf(Predicate<? super E> filter) {...}
  default Stream<E> stream() {...}
  default Stream<E> parallelStream() {...}
}
```

13 abstrakte Methoden

4 zusätzliche Default-Methoden

➔ Damit **erben** alle **implementierenden Klassen** diese **Default-Methoden** und müssen **nicht selbst erweitert** werden!

© H. Prähofer, 2026

Einschränkungen bei Default-Methoden

- Nur **public** Default-Methoden
- Eine Default-Methode kann nicht **final** deklariert werden.
- Eine Default-Methode darf keine public Methode von **Object** überschreiben. Das betrifft:
  - **toString()**
  - **equals(Object)**
  - **hashCode()**

© H. Prähofer, 2026

3. Interfaces

3.1 Motivation

3.2 Interfaces in Java

3.3 Default-Methoden

3.4 Statische und private Members

3.5 Beispiele aus der Bibliothek

3.6 Fallstudie List

© H. Prähofer, 2026

Statische Konstanten und Methoden

- Interfaces können **statische Konstanten** (nur **final**) und **statische Methoden** haben
  - sind über den **Namen des Interfaces zugreifbar**

```
public interface Writer {
  char EOL = '\n';

  static Writer fromFile(java.io.File file)  {...}

  ...
 }
```

statische Konstante
- automatisch **public static final**

statische Methode
- automatisch **public**

implizit

```
public interface Writer {
  public static final char EOL = '\n';

  public static Writer fromFile(java.io.File file)  {...}

  ...
 }
```

Verwendung:

Ansprechen über Namen des Interfaces

```
java.io.File file = new java.io.File(fileName);
Writer fileWriter = Writer.fromFile(file);
```

```
fileWriter.write(Writer.EOL);
```

© H. Prähofer, 2026

Private Methoden

Hilfsmethoden für Default-Methoden und/oder statische Methoden

Beispiele

private Methode **writeEOL**

```
public interface Writer {
  void write(char c);
  void open();
  void close();
  ...
  default void writeLn() {
    writeEOL();
  }

  default void writeLn(String s) {
    write(s);
    writeEOL();
  }

  private void writeEOL() {
    write('\n');
  }
}
```

verwendet abstrakte Methode

© H. Prähofer, 2026

3. Interfaces

3.1 Motivation

3.2 Interfaces in Java

3.3 Default-Methoden

3.4 Statische und private Members

3.5 Beispiele aus der Bibliothek

3.6 Fallstudie List

© H. Prähofer, 2026

Interface java.lang.Runnable

- Interface für ein Objekt, das man ausführen kann
  - Z.B.: für die Ausführung durch einen Thread

```
public interface Runnable {
  void run();
}
```

Beispiel: Implementierung einer Uhr

```
public class Clock extends Timer implements Runnable {
  public Clock (int h, int m, int s) { super(h, m, s); }

  public void run() {
    try {
      while (true) {
        this.add(1);
        System.out.println(this.toString());
        Thread.sleep(1000);  // wait 1 second
      }
    } catch (InterruptedException e) {}
  }
}
```

```
Thread thread = new Thread(new Clock(20, 15, 00));
thread.start();
```

Startet den Thread und ruft **run()** auf

© H. Prähofer, 2026

Iterator<T> und Iterable<T>

- **Iterator<T>**: zur Iteration über Collections

```
package java.util;

public interface Iterator<E> {
  boolean hasNext();
  E next();

  default void remove() {
    throw new UnsupportedOperationException("remove");
  }
  default void forEachRemaining(Consumer<? super E> action) {
    ...
  }
}
```

Schleifenkonstruktion mit **Iterator**:

```
Iterable<Object> iterable = ...

Iterator<Object> iterator = iterable.iterator();
while (iterator.hasNext()) {
  Object o = iterator.next();
  ...
}
```

- **Iterable<T>**: Interface für alle Typen die Iterator haben

```
package java.lang;
public interface Iterable<T> {
  Iterator<T> iterator();
  default void forEach(Consumer<? super T> action) {...}
  default Spliterator<T> spliterator() {...}
}
```

foreach-Schleife für alle **Iterable**s:

```
for (Object o: iterable) {
  ...
}
```

© H. Prähofer, 2026

Comparable<T>

- Für Ordnungsrelation

```
public interface Comparable<T> {
  public int compareTo(T other);
}
```

```
 < 0  : this < other
== 0  : this.equals(other)
 > 0  : this > other
```

- Implementierung durch Klassen

```
public class Person implements Comparable<Person> {
  private String firstName;
  private String lastName;
  private int age;
  ...

  public int compareTo(Person other) {
    int c = this.lastName.compareTo(other.lastName);
    if (c != 0) return c;
    c = this.firstName.compareTo(other.firstName);
    if (c != 0) return c;
    return Integer.compare(this.age, other.age);
  }
}
```

© H. Prähofer, 2026

Comparator<T>

- Zum Vergleich von zwei Objekten

```
package java.util;
public interface Comparator<T> {
  int compare(T o1, T o2);

  default Comparator<T> reversed() {
    return Collections.reverseOrder(this);
  }
  ...
}
```

mehrere Default-Methoden

- Erzeugen von Comparator-Objekten (unabhängig von eigentlicher Klasse)

```
import java.util.Comparator;

public class PersonComparator implements Comparator<Person> {
  public int compare(Person p1, Person p2) {
    int c = Integer.compare(p1.age(), p2.age());
    if (c != 0) return c;
    c = p1.getLastName().compareTo(p2.getLastName());
    if (c != 0) return c;
    return p1.getFirstName().compareTo(p2.getFirstName());
  }
}
```

Mehr zu Comparable und Comparator bei Kapitel Klassenbibliothek!

© H. Prähofer, 2026

Sonderfall: Tagging Interfaces

Achtung: Statt Tagging Interfaces werden heute Annotationen eingesetzt!

- Tagging Interfaces haben **keine Methode**
- werden ausschließlich zur **Kennzeichnung von Klassen** verwendet
  ➔ Klasse ist durch den **Interface-Typ** gekennzeichnet

Bespiele:

- **Serializable**: für Klassen, die über Serialisierungsmechanismus gespeichert werden können

```
package java.io;

public interface Serializable { }
```

- **Cloneable**: für Klassen, die man mit **clone()** kopieren können soll

```
package java.lang;

public interface Cloneable { }
```

© H. Prähofer, 2026

Fallbeispiel: Interface Cloneable und Methode clone

- Tagging-Interface **Cloneable** zur Kennzeichnung von Klassen, die **clone** unterstützen sollen

```
public interface Cloneable { }
```

- Methode **clone** in **Object**
  - native Methode für Speicherkopie des aufgerufenen Objekts
  - wirft eine **CloneNotSupportedException**, wenn die Klasse **Cloneable** nicht implementiert
  - ist **protected**, kann daher nur in Unterklassen aufgerufen werden
  - Rückgabetyp ist **Object**, zurückgegebenes Objekt ist vom gleichem Typ wie geclontes Objekt

```
public class Object {
  protected native Object clone() throws CloneNotSupportedException;
  ...
}
```

- Klasse, die **clone** unterstützen soll, muss
  - **Cloneable** implementieren
  - **clone** von **Object** überschreiben ➔ siehe nächste Folie

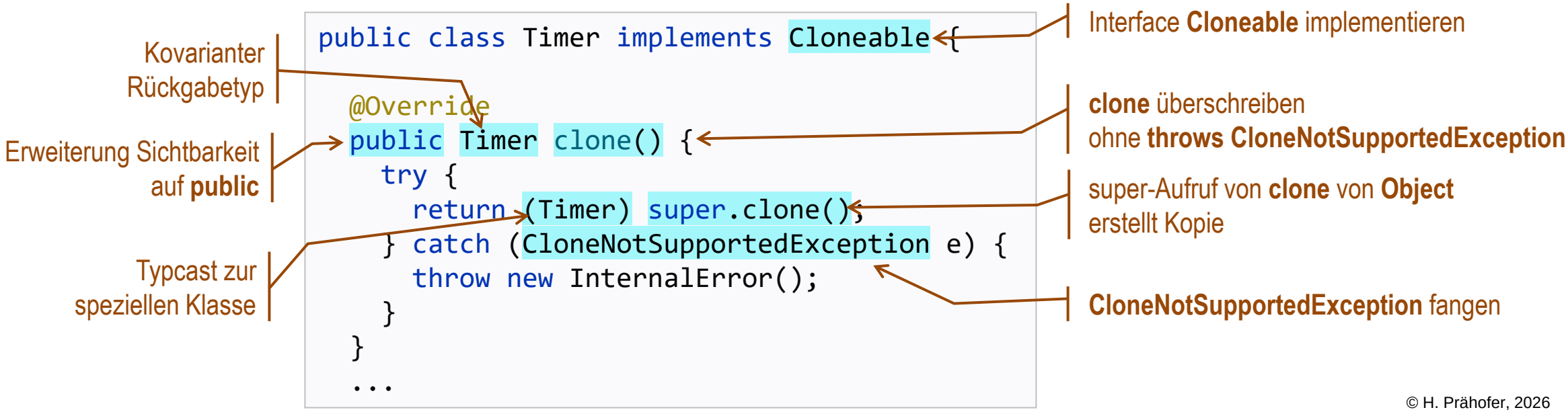
© H. Prähofer, 2026

Fallbeispiel: Interface Cloneable und Methode clone

Klasse, die clone unterstützen soll, muss clone von Object folgend überschreiben

- Sichtbarkeit auf **public** erweitern
- Typ des Rückgabewerts kovariant auf Klasse spezialisieren
- **CloneNotSupportedException** abfangen und keine Exception werfen
- mit Super-Call Kopie erstellen
- Kopie auf Klasse casten und zurückgeben

```
public class Object {
  protected native Object clone() throws CloneNotSupportedException;
```



© H. Prähofer, 2026

Alternative zu clone: Copy-Konstruktoren

- Als Alternative zu **clone**, werden heute oft Copy-Konstruktoren verwendet
  - Konstruktor mit dem zu kopierenden Objekt als Parameter
  - Initialisieren der Felder mit Werten von zu kopierendem Objekt
  - Möglichkeit mit Copy-Konstruktoren der Felder eine tiefe Kopie zu erstellen

```
public class Person {
  private final String name;
  private final Address address;

  public Person(String name, Address address) {
    this.name = name;
    this.address = address;
  }

  public Person(Person other) {
    this.name = other.name;
    this.address = new Address(other.address);
  }

  public String name() { return name; }

  public Address address() { return address; }
}
```

Copy-Konstruktor mit zu kopierendem Objekt als Parameter

Setzen der Werte aus kopiertem Objekt

Tiefe Kopie durch Copy-Konstruktor von Feldwert

© H. Prähofer, 2026

Lernziele überprüfen

- Worin unterscheiden sich Interfaces von Klassen?
- Welche Arten von Methoden können in Interfaces definiert werden?
- Wie wird ein Interface in Java deklariert und wie implementiert eine Klasse ein Interface?
- Was bedeutet es, dass Interfaces Mehrfachvererbung erlauben?
- Was sind Default-Methoden?
- Wie werden Default-Methode meistens implementiert?
- Was sind Tagging-Interfaces (Marker-Interfaces) und nenne Beispiele aus der Java-Standardbibliothek.

© H. Prähofer, 2026

3. Interfaces

3.1 Motivation

3.2 Interfaces in Java

3.3 Default-Methoden

3.4 Statische und private Members

3.5 Beispiele aus der Bibliothek

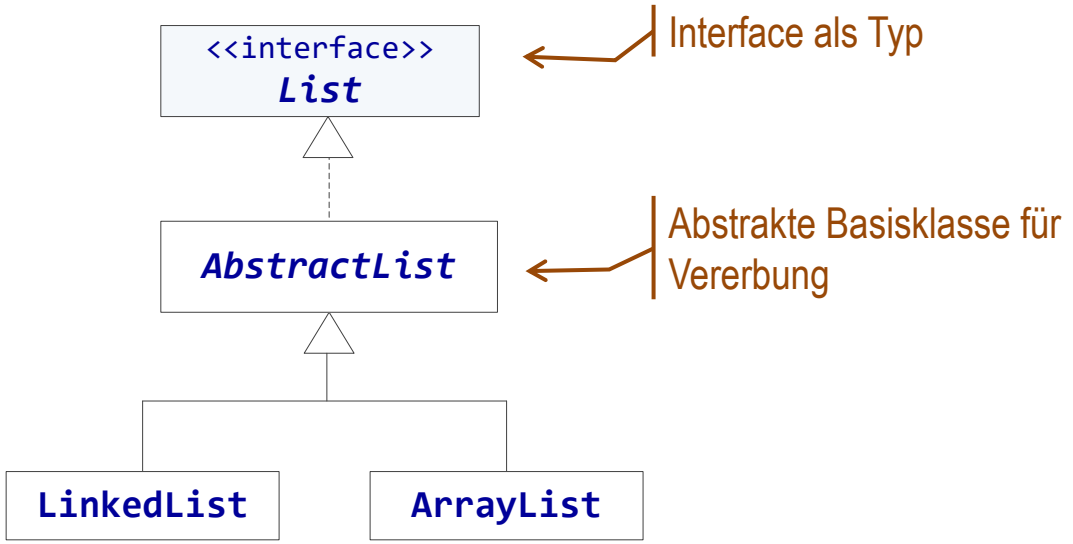
3.6 Fallstudie List

- ☐ Interface List
- ☐ Interfaces List und MutableList
- ☐ Vorschau

© H. Prähofer, 2026

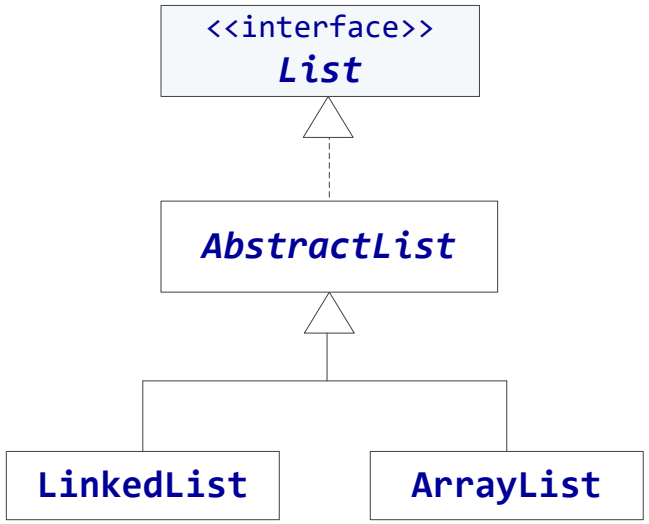
Interface List

- Interface **List**
  - fixiert Schnittstelle
  - wird als Typ verwendet
- Abstrakte Basisklasse **AbstractList**
  - für Gemeinsamkeiten in Implementierung
- Konkrete Klassen
  - **LinkedList**
  - **ArrayList**
  - ...



© H. Prähofer, 2026

Interface List




```
public interface List {
  public static List of(Object single) {...}
  public static List of(Object[] array) {...}

  int size();
  Object get(int idx);
  int indexOf(Object o);
  default boolean contains(Object o) {
    return indexOf(o) >= 0;
  }
  void clear();
  void add(Object elem);
  void add(int idx, Object elem);
  void addAll(Object... elems);
  Object remove(int idx);
  boolean remove(Object elem);
}
```

Schnittstelle

evtl. Default-Methoden

```
public abstract class AbstractList implements List {
  private int size = 0;
  public int size() {
    return size;
  }
  ...
}
```

Gemeinsame Daten
Gemeinsame Methoden

```
public class LinkedList extends AbstractList {
  protected Node head, tail;
  ...
}
```

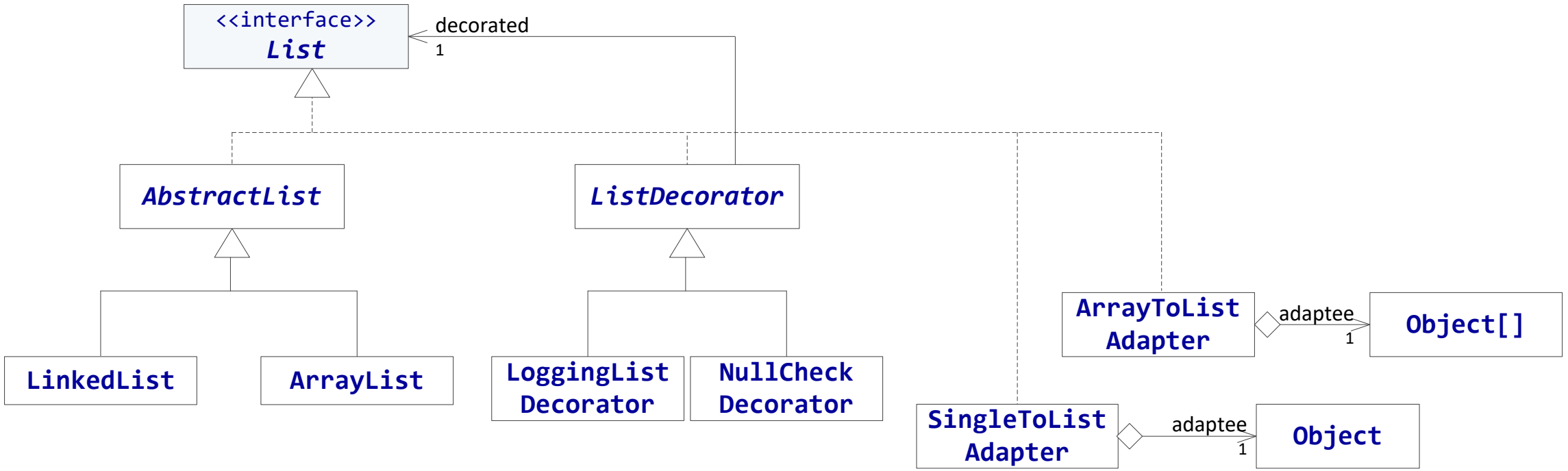
```
public class ArrayList extends AbstractList {
  protected Object[] elems;
  ...
}
```

konkrete Klassen

© H. Prähofer, 2026

Interface List

- **Decorator** and **Adapter** implementieren **List** Interface



© H. Prähofer, 2026

3. Interfaces

3.1 Motivation

3.2 Interfaces in Java

3.3 Default-Methoden

3.4 Statische und private Members

3.5 Beispiele aus der Bibliothek

3.6 Fallstudie List

- ☐ Interface List
- ☐ Interfaces List und MutableList
- ☐ Vorschau

© H. Prähofer, 2026

UnmodifiableListDecorator: Problemanalyse

UnmodifiableListDecorator wirft UnsupportedOperationException

- Decorator, um Veränderungen zu verhindern

```
public class UnmodifiableListDecorator extends ListDecorator {

  public UnmodifiableListDecorator(List decorated) {
    super(decorated);
  }

  @Override
  public void add(Object elem) {
    throw new UnsupportedOperationException("forbidden");
  }

  @Override
  public void add(int idx, Object elem) {
    throw new UnsupportedOperationException("forbidden");
  }

  @Override
  public Object remove(int idx) {
    throw new UnsupportedOperationException("forbidden");
  }

  ...
}
```

Anwendung: **nur lesender** Zugriff auf interne Daten

```
public class MyClass {

    private List myPrivateListData = new ArrayList();

    public List getListData() {
        return new UnmodifiableListDecorator(myPrivateListData);
    }
    ...
}
```

Nur lesende Zugriffe !

```
MyClass myObject = new MyClass();
List data = myObject.getListData();

data.remove(...); // throws UnsupportedOperationException
```

Problematisch: Interface **List** bietet **add**, **remove** etc. an, und **UnmodifiableList** verhindert dies mit einer Exception!?

© H. Prähofer, 2026

Interfaces List und MutableList

Approach von Kotlin!

- Trennung der Interfaces
  - **List**: nur lesende Zugriffe (*read-only*)
  - **MutableList**: Methoden mit Veränderungen

```
public interface List {
  public int size();
  public Object get(int idx);
  public default boolean contains(Object o) {
    return indexOf(o) >= 0;
  }
  public int indexOf(Object o);
}
```



```
public interface MutableList extends List {
  public void add(Object elem);
  public default void addAll(Object...elems) {
    for (Object elem: elems) { add(elem); }
  }
  public boolean remove(int idx);
  public default boolean remove(Object elem) {
    return remove(indexOf(elem));
  }
  public default void clear() {...}
}
```

```
public class LinkedList implements MutableList { ... }
```

```
public class ArrayList implements MutableList { ...}
```

Verwendung bei Datenkapselung:

```
public class MyApplication {
    private MutableList myPrivateListData = new ArrayList();
    public void addData(String data) {
        myPrivateListData.add(data);

    public List getListData() {
        return myPrivateListData;
    }
}
```

Rückgabe als read-only Interface

intern als mutable

Problem: mit Typecast kann Anwender Zugriffsschutz umgehen

```
MyClass myObject = new MyClass();
List list = myObject.getListData();
MutableList mutList = (MutableList) list;
mutList.clear();
```

Client kann mit Typcast vollen Zugriff erlangen

© H. Prähofer, 2026

Wirklich unveränderliche Liste

- Wrapper, der nur **List** implementiert

```
public class UnmodifiableListWrapper implements List {

    private final List list;

    public UnmodifiableListWrapper(List list) {
        this.list = list;
    }
    @Override
    public int size() {
        return list.size();
    }
    @Override
    public Object get(int idx) {
        return list.get(idx);
    }
    @Override
    public boolean contains(Object o) {
        return list.contains(o);
    }
    @Override
    public int indexOf(Object o) {
        return list.indexOf(o);
    }
 }
```

```
public class MyClass {
    private List myPrivateListData = new ArrayList();

    public List getListData() {
        return new UnmodifiableListWrapper(myPrivateListData);
    }
    ...
}
```

Rückgabe als read-only Interface

Wrapper mit nur lesenden Zugriffen !

Typecast nicht mehr möglich

```
MyClass dataClass = new MyClass();

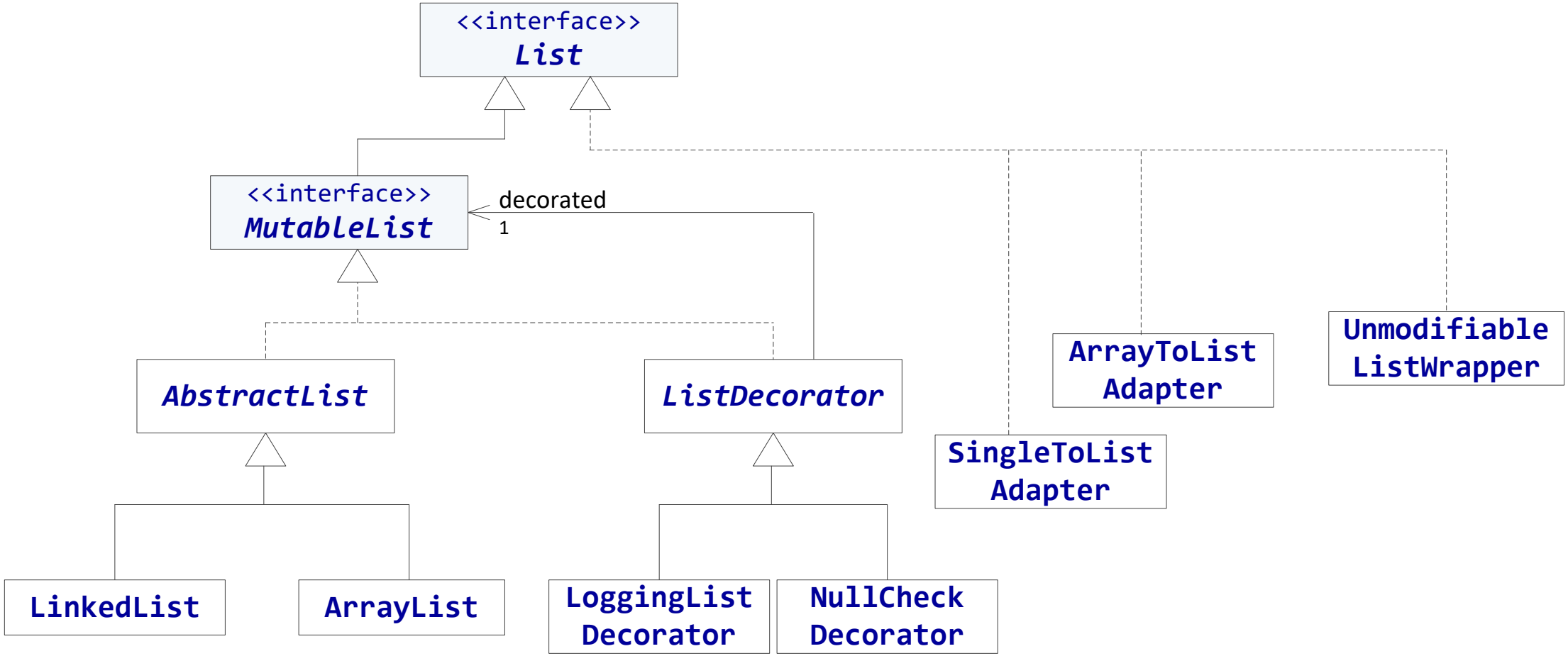
List list = dataClass.getListData();
```

```
Mutable mutList = (MutableList) list;
```

✗

ClassCastException !

Klassendiagramm List und MutableList



© H. Prähofer, 2026

3. Interfaces

3.1 Motivation

3.2 Interfaces in Java

3.3 Default-Methoden

3.4 Statische und private Members

3.5 Beispiele aus der Bibliothek

3.6 Fallstudie List

- ☐ Interface List
- ☐ Interfaces List und MutableList
- ☐ Vorschau

© H. Prähofer, 2026

Weitere Verbesserungen

- Innere Klassen für Knoten ➔ 5. Innere Klasse
- Iterator für sequentiellen Zugriff ➔ 5. Innere Klasse
- Generische Elementtypen ➔ 6. Generics

© H. Prähofer, 2026